به نام خدا

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

هوش مصنوعی قابل اعتماد

تمرین دوم

نام و نام خانوادگی	امین آقاکثیری
شماره دانشجویی	810101381
تاریخ ارسال گزارش	1405/3/3

فهرست

سوال اول: تفسیر پذیری 3

بخش اول: سوالات توضیحی 3

زیربخش اول: PDP 3

زیربخش دوم: LIME 4

زیربخش سوم: SHAPLEY VALUES 4

زیربخش چهارم: NAM 5

بخش دوم: سوالات پیاده سازی 7

زیربخش اول: مقداردهی مدل 7

زیربخش دوم: بخش بندی تصویر 7

زیربخش سوم: تولید نمونه های تغییر یافته 7

زیربخش چهارم: وزن دهی تصاویر تغییر یافته 8

زیربخش پنجم: آموزش مدل خطی 8

زیربخش ششم: تصویر خرس قطبی 10

سوال دوم: توصیف پذیری مدل های هوش مصنوعی با GRAD-CAM 11

بخش اول: GRAD-CAM 11

زیربخش اول: پرسش و پاسخ 11

زیربخش دوم: پیاده سازی 12

بخش دوم: FINERGRAD-CAM 15

زیربخش اول: پرسش و پاسخ 15

زیربخش دوم: پیاده سازی 17

بخش سوم: مقایسه 19

زیربخش اول: مقایسه دو روش 19

مراجع 21

سوال اول: تفسیر پذیری

بخش اول: سوالات توضیحی

زیربخش اول: PDP

پاسخ بخش الف: اگر همبستگی زیادی بین اینها وجود داشته باشد، مدل باید برای پیش بینی مقادیر خارج از فضای داده های آموزشی و محاسبه نمودار PDP از extrapolation استفاده کند زیرا در فرمول PDP، مدل بر روی توزیع marginal ویژگی های C میانگین گیری می کند و این یعنی ممکن است ترکیب های جدید برای S و C درون داده های آموزشی ما وجود نداشته باشند و اینجا مجبور به extrapolation می شویم که برای مدل های جعبه سیاه ممکنه عملکرد خوبی نداشته باشد.

پاسخ بخش ب: در روش PDP معمولی یک یا چند ویژگی که در S هستند را ثابت نگه می داریم و هر بار با سایر ویژگی ها ترکیب می کنیم (و این ویژگی های درون S را هر دفعه تغییر می دهیم به یک مقدار جدیدی و دوباره آن را ثابت نگه می داریم) و این همانطور که در قسمت قبل گفتم ممکنه باعث تولید داده های جدیدی بشه که اصلا شبیه داده های آموزشی نیستند و باعث extrapolation بشه و نتونیم تاثیر واقعی آن ویژگی ها را بفهمیم. در ALE به جای بررسی اثر کلی یک ویژگی با ساختن ترکیب های مصنوعی، اثر محلی ویژگی را فقط در نواحی ای محاسبه می کند که واقعا داده در آن ها وجود دارد. در این روش به جای استفاده از توزیع marginal، از توزیع شرطی ویژگی های مکمل استفاده می کند تا از extrapolation نا معتبر جلوگیری کند و در واقع فقط مقادیری از ویژگی های دیگر را استفاده می کند که با مقدار فعلی ویژگی موردنظر در داده ها واقعا مشاهده شده اند. به این صورت که ابتدا تاثیرات محلی (مشتق جزئی تابع مدل نسبت به ویژگی مورد نظر) را در یک همسایگی کوچک محاسبه می کند و سپس این تاثیرات محلی را صرفا با توجه به توزیع شرطی ویژگی های مکمل در همان همسایگی میانگین گیری می کند. در نهایت، این تاثیرات محلیِ میانگین گیری شده را در طول دامنه ویژگی با استفاده از انتگرال روی هم انباشته می کند تا تاثیر کلی آن ویژگی به دست آید. این روش باعث می شود که ارزیابی مدل تنها در مناطقی انجام شود که داده های آموزشی در آنجا حضور داشتند.

پاسخ قسمت ج: خیر به صورت کلی نمی شود این نتیجه را گرفت. این خط افقی فقط نشان می دهد که وقتی اثر X_1 را به صورت میانگین روی کل توزیع ویژگی های دیگر بررسی می کنیم، میانگین پیش بینی مدل با تغییر X_1 تغییر زیادی نمی کند. اما این به معنای بی اثر بودن کامل X_1 در خود مدل نیست. یک حالت ممکن است این باشد که اثر X_1 به شکل تعاملی باشد. یعنی اثر X_1 به مقدار ویژگی های دیگر بستگی داشته باشد. در این حالت ممکن است برای بعضی نمونه ها افزایش X_1 خروجی مدل را زیاد کند و برای بعضی نمونه های دیگر خروجی را کم کند. وقتی PDP از این اثرها میانگین می گیرد، اثر های مثبت و منفی می توانند همدیگر را خنثی کنند و نمودار نهایی صاف و افقی شود در حالی که مدل واقعا از X_1 استفاده می کند. حالت دیگر ممکن است این باشد که اگر X_1 با ویژگی های دیگر همبستگی داشته باشد، PDP ممکن است برای محاسبه اثر آن از ترکیب های غیرواقعی ویژگی ها استفاده کند. در چنین شرایطی شکل PDP می تواند گمراه کننده باشد و لزوما نشان دهنده نقش واقعی X_1 در مدل نباشد.

زیربخش دوم: LIME

مشکل زمانی ایجاد می شود که مدل جعبه سیاه در اطراف آن نقطه رفتار بسیار ناپایدار یا نویزی داشته باشد و یا مشتق ناپذیر باشد. یعنی حتی با تغییرات بسیار کوچک در ورودی، خروجی مدل به مقدار زیادی تغییر کند. در چنین حالتی، اگر اندازه همسایگی در LIME خیلی کوچک انتخاب شود، ممکن است نمونه های تولید شده فقط بخش بسیار محدودی از رفتار مدل را ببینند و مدل خطی به دست آمده تصادفی و وابسته به نویز شود. از طرف دیگر، اگر همسایگی کمی بزرگ تر انتخاب شود، ممکن است وارد ناحیه ای شویم که رفتار مدل دیگر خطی نیست و یک مدل خطی ساده نتواند آن را درست تقریب بزند. تقریب خطی ذاتا بر پایه مشتق است و فرض می کند که تابع در یک همسایگی کوچک رفتار ملایمی دارد. طبق تعریف یک تابع L-Lipschitz است اگر و تنها اگر در تمام نقاط مشتق پذیر باشد و مشتق اش هم از L کوچک تر باشد. حال اگر L مدل ما خیلی بزرگ باشد، پس با یک تغییر بسیار کوچک در ورودی، خروجی بسیار تغییر می کند و تقریب خطی در اینجا دقیق نیست و همچنین اگر کلا در یک ناحیه مشتق پذیر نباشد هم این مشکل وجود دارد. برای اینکه روش LIME در یک همسایگی مشخص معتبر باشد، تابع مدل ما باید حداقل به صورت محلی با L کوچک، Lipschitz باشد. این ویژگی تضمین می‌کند که تابع به اندازه کافی هموار است و تقریب خطی می تواند با خطای کمی رفتار مدل را در آن همسایگی شبیه سازی کند.

زیربخش سوم: SHAPLEY VALUES

پاسخ بخش الف: پیش بینی مدل برای نمونه x^i برابر است با:

$$f(x^i) = \beta_0 + \beta_1 x_1^i + \cdots + \beta_p x_p^i$$

از طرف دیگر، میانگین پیش بینی های مدل روی کل داده ها برابر است با میانگین همین عبارت. چون β_0 و ضرایب β_j ثابت هستند، داریم:

$$E[f(X)] = \beta_0 + \beta_1 E[X_1] + \cdots + \beta_p E[X_p]$$

حال اگر اختلاف پیش بینی نمونه خاص با میانگین پیش بینی ها را حساب کنیم، به همان رابطه 2 می رسیم:

$$f(x^i) - E[f(X)] = \beta_1(x_1^i - E[X_1]) + \cdots + \beta_p(x_p^i - E[X_p])$$

این اختلاف نشان می دهد که پیش بینی نمونه x^i چقدر از پیش بینی متوسط مدل بیشتر یا کمتر است. مشاهده می کنیم که سهم هر ویژگی برابر می شود با ضریب آن ویژگی ضربدر اختلاف مقدار آن ویژگی در نمونه موردنظر با مقدار متوسط همان ویژگی در داده ها. همانطور که در بالا می بینیم، در این مدل خطی هیچ گونه اثر متقابل ای بین ویژگی ها وجود ندارد و اثرات آن‌ها کاملا جمع پذیر اند. این یعنی هر ویژگی به‌طور کاملا مستقل بخشی از این اختلاف کل را تولید می‌کند. بر اساس مفهوم مقادیر Shapley که تقسیم ها و سهم ها به صورت عادلانه هستند، وقتی یک ویژگی مانند j به طور مستقل مقدار $\beta_j(x_j^i - E[X_j])$ را به تغییرات خروجی اضافه می کند، سهم عادلانه آن (که همان Shapley Value است) هم برابر با همان مقدار خواهد بود.

پاسخ بخش ب: با توجه به بخش الف، برای حل این مشکل می توان به جای محاسبه دقیق Shapley روی خود مدل جعبه سیاه، ابتدا رفتار مدل جعبه سیاه را با یک مدل خطی ساده تر تقریب بزنیم و سپس Shapley را برای آن مدل خطی محاسبه کنیم. برای مثال، می توانیم در اطراف نمونه موردنظر چندین نمونه مصنوعی یا واقعی انتخاب کنیم، خروجی مدل جعبه سیاه را برای آن ها به دست آوریم و سپس یک مدل خطی قابل تفسیر آموزش دهیم که خروجی آن تا حد ممکن شبیه خروجی مدل جعبه سیاه در همان ناحیه باشد. بعد از اینکه این مدل خطی ساخته شد، دیگر نیازی به بررسی همه زیرمجموعه های ویژگی ها نیست زیرا طبق بخش الف، سهم هر ویژگی در مدل خطی برابر است با ضریب آن ویژگی ضربدر اختلاف مقدار آن ویژگی با میانگین آن.

زیربخش چهارم: NAM

ایده NAM بر اساس ایده مدل های Generalized Additive Models (GAMs) است. در یک شبکه عصبی استاندارد، تمامی ویژگی های ورودی در لایه های پنهان با یکدیگر ترکیب می شوند و این موضوع درک تاثیر هر ویژگیِ را سخت می کند. در معماری این مدل، خروجی نهایی به صورت جمع اثر های جداگانه هر ویژگی ساخته می شود. یعنی مدل به جای اینکه همه ویژگی ها را هم زمان وارد یک شبکه عصبی بزرگ کند، برای هر ویژگی x_i یک شبکه عصبی جداگانه یاد می گیرد که فقط همان ویژگی را به عنوان ورودی می گیرد و

یک تابع تک متغیره $f_i(x_i)$ تولید می کند. سپس خروجی این شبکه های جداگانه با هم جمع می شوند و همراه با یک بایاس، پیش بینی نهایی را می سازند. در واقع NAM ها یک ترکیب خطی از شبکه هایی را یاد می گیرند که هر کدام فقط به یک ویژگی ورودی توجه می کنند و این شبکه ها به صورت مشترک آموزش داده می شوند. این معماری به تفسیرپذیری کمک می کند چون سهم هر ویژگی به صورت مستقل در یک تابع جداگانه نمایش داده می شود. برای تفسیر مدل کافی است نمودار $f_i(x_i)$ نسبت به x_i را رسم کنیم. این نمودار نشان می دهد که با تغییر مقدار یک ویژگی، سهم آن ویژگی در پیش بینی چگونه تغییر می کند.

مزیت NAM نسبت به مدل خطی این است که مدل خطی فقط می تواند اثر هر ویژگی را به صورت یک خط مستقیم مدل کند. یعنی در مدل خطی، اگر ضریب یک ویژگی مثبت باشد، با افزایش آن ویژگی خروجی همیشه به صورت خطی افزایش پیدا می کند و اگر ضریب منفی باشد همیشه به صورت خطی کاهش پیدا می کند. اما در بسیاری از مسائل واقعی، اثر ویژگی ها خطی نیست. NAM می تواند برای هر ویژگی یک تابع غیر خطی پیچیده یاد بگیرد زیرا هر f_i با یک شبکه عصبی مدل می شود. پس NAM نسبت به مدل خطی انعطاف پذیر تر و دقیق تر است.

محدودیت اصلی NAM این است که ساختار آن جمع پذیر است. یعنی فرض می کند اثر کل ویژگی ها را می توان به صورت جمع اثر های جداگانه هر ویژگی نوشت. بنابراین در حالت پایه، NAM ارتباط های پیچیده بین ویژگی ها را به طور مستقیم مدل نمی کند.

در رابطه با مشکل PDP، NAM تا حدی این مشکل را کاهش می دهد اما کاملا حل نمی کند. درNAM ، نمودار هر ویژگی نتیجه میانگین گیری پس از آموزش روی ترکیب های مصنوعی نیست بلکه خود مدل از ابتدا به صورتی ساخته شده که اثر هر ویژگی را به صورت یک تابع جداگانه یاد بگیرد. بنابراین برای تفسیر اثر یک ویژگی لازم نیست مانند PDP مقدار آن ویژگی را با همه حالت های ممکن ویژگی های دیگر ترکیب کنیم و نمودار $f_i(x_i)$ مستقیما سهم همان ویژگی در محاسبه مدل را نشان می دهد. اما NAM کاملا این مشکل را حل نمی کند. اگر در یک بازه از یک ویژگی داده آموزشی کم باشد، تابع یادگرفته شده در آن ناحیه می تواند نامطمئن تر باشد. برای همین مقاله می گوید که هنگام رسم نمودار هایNAM ، چگالی داده ها هم روی همان نمودار نشان داده شود تا مشخص باشد مدل در کدام نواحی داده کافی برای یادگیری داشته و در کدام نواحی نداشته.

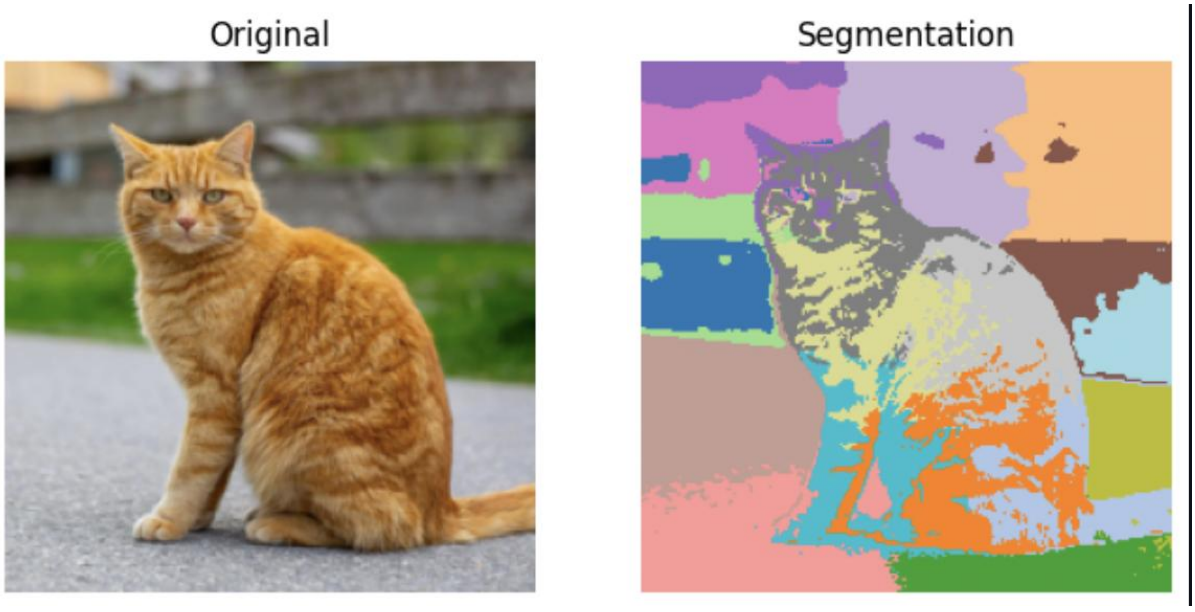
بخش دوم: سوالات پیاده سازی

زیربخش اول: مقداردهی مدل

اندیس 282 که مربوط به اندیس لیبل درست برای گربه بود، احتمال 90.55 درصد را به عنوان بالاترین احتمال داشت.

زیربخش دوم: بخش بندی تصویر

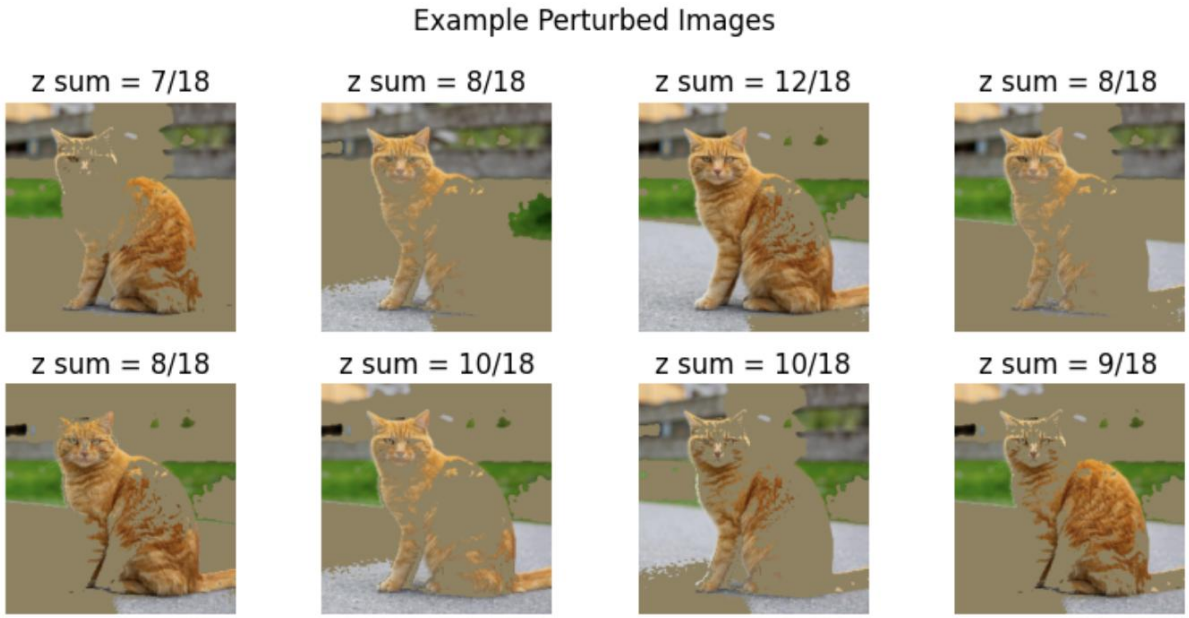
در تابع داده شده برای بخش بندی تصاویر از الگوریتم کلاسترینگ k-means استفاده شده است. در این روش، هر پیکسل تصویر به عنوان یک نقطه در فضای ویژگی در نظر گرفته شده. ویژگی های هر پیکسل شامل مقادیر RGB و همچنین مختصات مکانی پیکسل در تصویر است. اضافه کردن مختصات مکانی باعث می شود پیکسل هایی که از نظر رنگ شبیه هستند ولی در بخش های کاملا متفاوت تصویر قرار دارند، الزاما در یک بخش قرار نگیرند. سپس k-means روی این ویژگی ها اجرا شده (با انتخاب مرکز برای هر دسته در هر تکرار و میانگین گیری و ست کردن هر مجموعه از طریق کمترین فاصله و تکرار این کار) و پیکسل ها به 18 خوشه تقسیم می شوند. تصویر زیر خوشه بندی برای عکس گربه است:



شکل 1. طبقه بندی عکس گربه به 18 قسمت

زیربخش سوم: تولید نمونه های تغییر یافته

آرایه 1000 تایی 18 بعدی تولید شد و برای هر یک از اینها با توجه به 1 یا 0 بودن هر درایه، 1000 تصویر تولید شد که 8 نمونه از آن ها به صورت زیر هستند:



شکل 2. نمونه تصاویر تغییر یافته

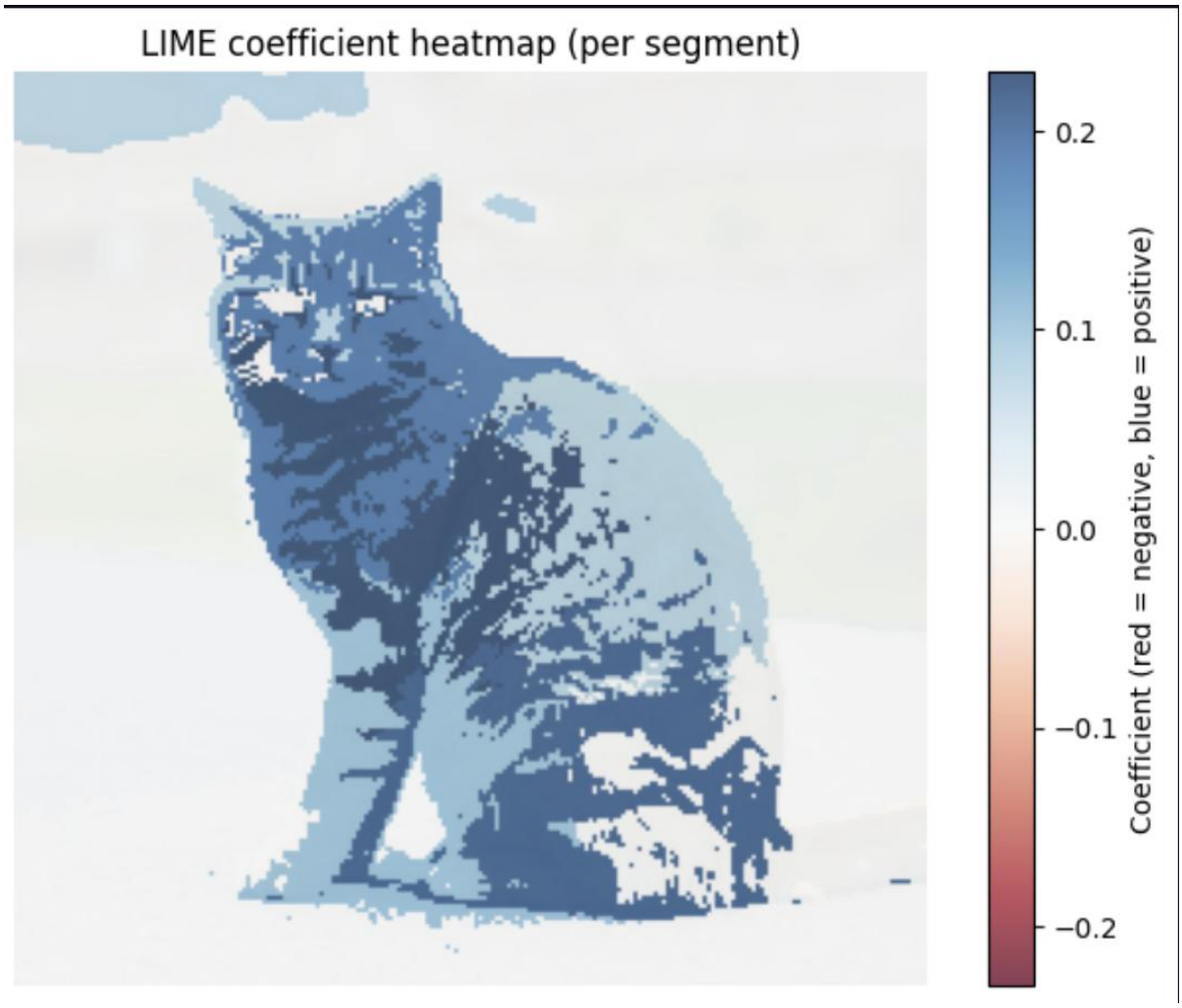
زیربخش چهارم: وزن دهی تصاویر تغییر یافته

در این بخش، برای هر تصویر تغییریافته یک وزن محاسبه شد تا مشخص شود آن نمونه چقدر به تصویر اصلی نزدیک است. در LIME، نمونه هایی که به تصویر اصلی شباهت بیشتری دارند باید تاثیر بیشتری در آموزش مدل خطی محلی داشته باشند، چون هدف LIME توضیح رفتار مدل در اطراف همان تصویر اصلی است، نه در کل فضای ورودی. برای این کار، هر تصویر تغییریافته با بردار باینری z نمایش داده شد و تصویر اصلی نیز با برداری شامل تمام مقدارهای 1 در نظر گرفته شد (چون تمام سگمنت ها را دارد). سپس فاصله کسینوسی بین z و بردار تصویر اصلی محاسبه شد. هرچه تعداد بخش های حفظ شده در تصویر تغییریافته بیشتر باشد، این فاصله کمتر و شباهت آن به تصویر اصلی بیشتر است (و وزن بیشتری می گیرد). در نهایت با استفاده از تابع نمایی داده شده، فاصله ها به وزن تبدیل شدند (که مقداری بین 0 و 1 است). بنابراین نمونه های نزدیک تر به تصویر اصلی وزن بزرگ تری گرفتند و در آموزش مدل خطی محلی اهمیت بیشتری خواهند داشت.

زیربخش پنجم: آموزش مدل خطی

در این بخش، ابتدا تمام تصاویر تغییریافته به مدل اصلی ResNet18 داده شدند و احتمال مربوط به کلاس 282 برای هر تصویر ذخیره شد. سپس از کد باینری هر تصویر تغییریافته به عنوان مدل خطی ای که می خواستیم آموزش دهیم استفاده شد و احتمال خروجی مدل اصلی برای همان تصاویر به‌عنوان مقدار هدف در نظر گرفته شد. این مدل خطی تلاش می‌کند در اطراف تصویر اصلی، رفتار مدل ResNet18 را تقریب بزند. در آموزش مدل خطی از وزن‌های محاسبه‌شده در بخش قبل استفاده شد. پس نمونه هایی که به تصویر اصلی نزدیک تر بودند تاثیر بیشتری روی آموزش مدل داشتند. همچنین از L1 Regularization استفاده شد تا

بسیاری از ضرایب کم اهمیت، صفر یا نزدیک به صفر شوند. این کار باعث می شود توضیح نهایی ساده تر و قابل فهم تر باشد و فقط بخش هایی از تصویر که واقعا روی تصمیم مدل اثر بیشتری دارند برجسته شوند. همچین درباره ضرایب نهایی، ضریب مثبت برای یک بخش به این معناست که حضور آن بخش احتمال کلاس 282 را افزایش می دهد، بنابراین آن بخش به نفع تشخیص کلاس موردنظر عمل کرده است. ضریب منفی یعنی حضور آن بخش احتمال کلاس 282 را کاهش داده است. Heatmap بدست آمده برا ضرایب نهایی مدل به صورت زیر شد:

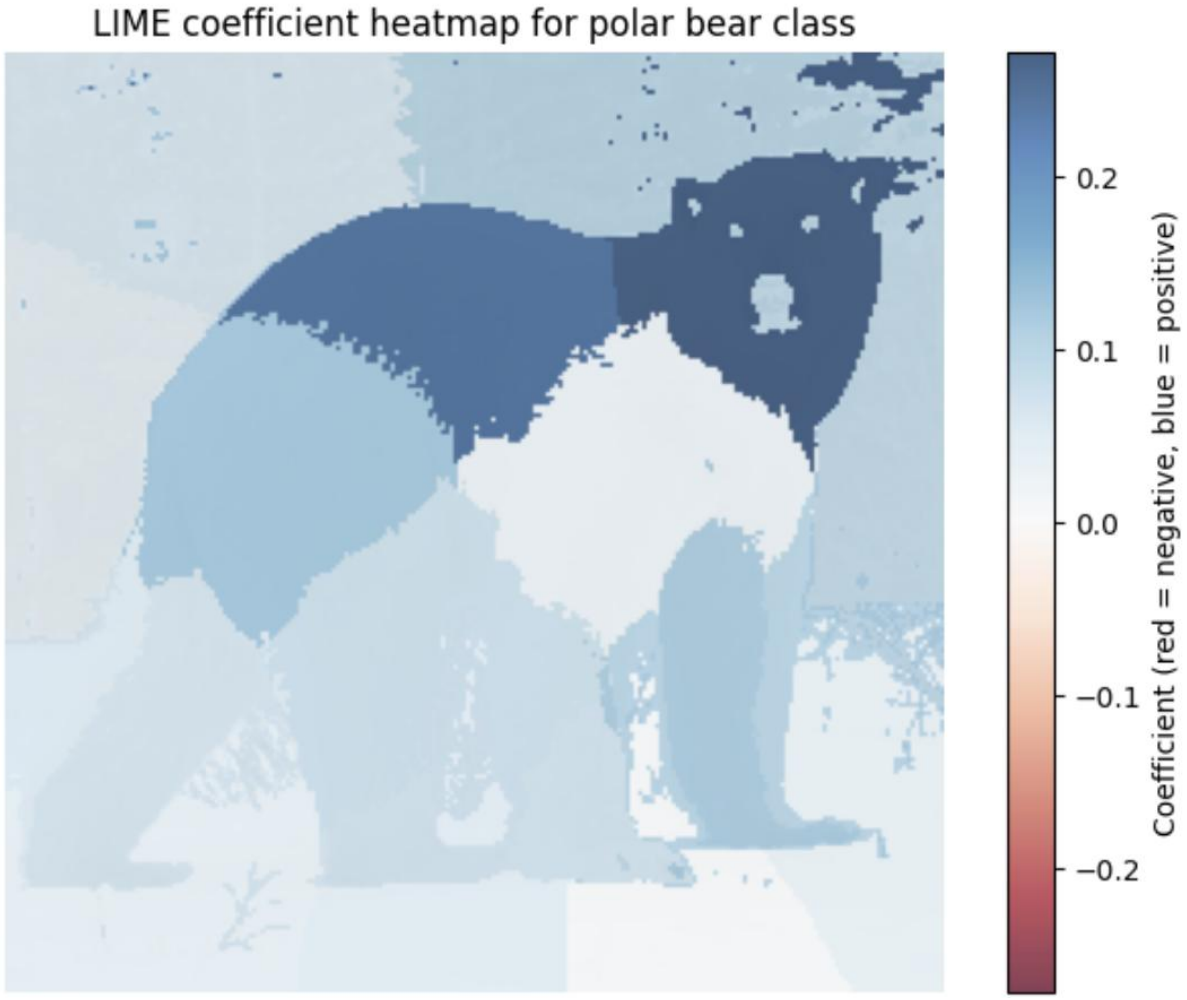


شکل 3. تصویر heatmap مربوط به اهمیت هر یک از 18 سگمنت برای گربه

در heatmap ، بخش های آبی اثر مثبت و بخش های قرمز (که در اینجا نداریم) اثر منفی دارند. هرچه شدت رنگ بیشتر باشد، اثر آن بخش بر تصمیم مدل قوی تر است. همانطور که در تصویر بالا دیده می شود، سمت سر و گردن گربه و پایین تنه آن و همچنین خط های روی پوست آن بیشترین اهمیت را برای مدل داشته است و پس زمینه مدل اهمیت زیادی نداشته و ضرایب مربوط به آن بسیار نزدیک به صفر هستند.

زیربخش ششم: تصویر خرس قطبی

مشابه روند بخش قبل، آن را برای این عکس هم تکرار کردم و بخش بندی کردم و سپس وزن ها را بدست آوردم و در نهایت مدل خطی را آموزش دادم. شکل heatmap آن در نهایت به صورت زیر شد:



شکل 4. تصویر heatmap مربوط به اهمیت هر یک از 18 سگمنت برای خرس قطبی

تفاوت اصلی این heatmap با heatmap مربوط به گربه این است که در تصویر گربه، جزئیات ظاهری مثل صورت، بدن و خطوط بدن نقش بیشتری در heatmap دارند، اما در تصویر خرس قطبی علاوه بر بدن و سر، بخش هایی از پس زمینه برفی و روشن اطراف هم تا حدی اثر مثبت نشان می دهند. این می تواند به این دلیل باشد که در تصاویر آموزشی خرس قطبی، محیط برفی و روشن معمولا همراه با خود خرس دیده می شود و مدل ممکن است از این پس زمینه به عنوان نشانه کمکی برای تشخیص کلاس خرس قطبی استفاده کرده باشد. با این حال، چون بیشترین ضرایب مثبت همچنان روی ناحیه خود خرس قرار دارند، می توان گفت تصمیم مدل بیشتر بر اساس خود شی اصلی گرفته شده است نه فقط پس زمینه. بنابراین خروجی

LIME برای هر دو تصویر منطقی است و نشان می دهد مدل برای هر کلاس به نواحی مرتبط با همان شی توجه کرده است.

سوال دوم: توصیف پذیری مدل های هوش مصنوعی با GRAD-CAM

بخش اول: GRAD-CAM

زیربخش اول: پرسش و پاسخ

پاسخ سوال 1: grad-cam بر اساس ایده استخراج گرادیان های یک کلاس مشخص نسبت به feature map های خروجی از cnn عمل می کند. همچنین این روش یک تعمیم از cam است زیرا cam فقط روی شبکه های با ساختار خاصی (که خروجی فقط logit باشد نه تسک های VQA و یا image captioning) آن هم با تغییر خود شبکه کار می کرد. در حالی که grad-cam می‌تواند برای هر شبکه کانولوشنی به کار رود و محدود به لایه آخر نیست و نیازی به تغییر در شبکه هم ندارد. تفاوت اصلی در این است که grad-cam با استفاده از گرادیان ها اهمیت هر کانال ویژگی را برای تصمیم نهایی مدل مشخص می کند. پیامد عملی این موضوع این است که می توانیم مناطقی از تصویر را که بیشترین تاثیر را در پیش بینی یک کلاس خاص دارند شناسایی کنیم (حتی اگر شبکه دارای ساختار پیچیده یا چند لایه غیرخطی باشد) بدون اینکه نیازی به تغییر خود شبکه اصلی داشته باشیم. این کار باعث می شود تفسیرپذیری افزایش یابد و به کاربر نشان دهد که مدل بر اساس کدام بخش های تصویر تصمیم گیری کرده است.

پاسخ سوال 2: می خواهند فقط تاثیرات مثبت را نگه دارند. زیرا گرادیان های منفی به این معنا هستند که افزایش مقدار آن ویژگی باعث کاهش احتمال کلاس هدف می شود و اثر منفی دارند، نه اثر تقویت کننده و معمولا مربوط به سایر کلاس ها هستند نه آن کلاس خاص که ما می خواهیم. اگر relu را حذف کنیم، خروجی نهایی شامل تاثیرات منفی هم می شود و تصویری که تولید می کنیم ممکن است مناطق مهم واقعی برای تقویت کلاس هدف را نتواند به درستی نشان دهد. بنابراین استفاده از relu باعث می شود grad-cam تنها مناطقی را نشان دهد که به صورت مثبت روی پیش بینی کلاس هدف اثر می گذارند و تفسیر آن برای کاربر قابل اعتماد تر شود.

پاسخ سوال 3: در آزمایش های مقاله، نویسندگان نشان دادند که grad-cam می تواند bias موجود در داده های آموزشی را با آشکار کردن ناحیه ای که مدل برای تصمیم گیری به آن توجه می کند مشخص کند. آن ها یک مدل را برای تشخیص دکتر و پرستار آموزش دادند و با وجود عملکرد ظاهرا خوب مدل روی داده های

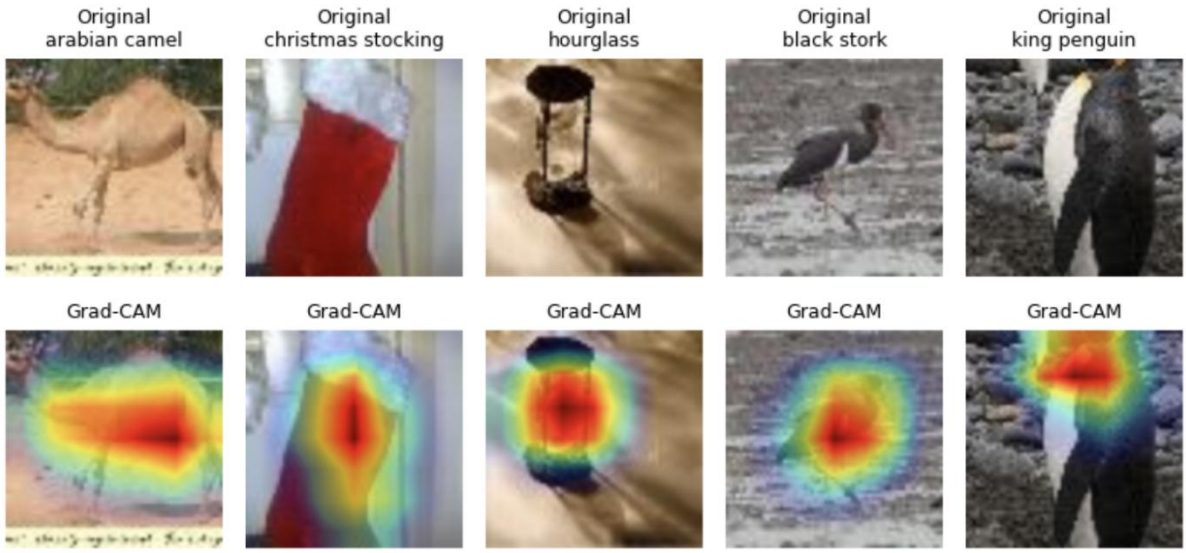
validation، grad-cam نشان داد که مدل برای تصمیم گیری به جای توجه به نشانه های مرتبط با شغل مانند روپوش سفید یا گوشی پزشکی، بیشتر به چهره، مدل مو و جنسیت افراد توجه می کند. این موضوع نشان داد که داده های آموزشی bias جنسیتی داشته اند. زیرا در داده های جمع‌آوری شده، بیشتر تصاویر دکتر ها مربوط به مردان و بیشتر تصاویر پرستار ها مربوط به زنان بوده است. بعد از این کشف، نویسندگان با افزودن داده های متوازن تر، مانند پرستاران مرد و پزشکان زن، مدل را دوباره آموزش دادند و مشاهده کردند که مدل جدید هم بهتر تعمیم می دهد و هم در grad-cam به نواحی درست تری مانند روپوش و گوشی پزشکی توجه می کند. اهمیت فنی این کار در این است که grad-cam فقط خروجی مدل را نشان نمی دهد بلکه منطق مدل را هم برای تصمیم گیری نشان می دهد و کمک می کند خطاهای پنهان داده و مدل شناسایی شوند. از نظر اخلاقی و اجتماعی هم اگر چنین bias هایی کشف نشوند، مدل ها می توانند دچار تبعیض های نژادی و جنسیتی شوند و درست تصمیم گیری نکنند.

پاسخ سوال 4: روش guided grad-cam ترکیبی از دو روش grad-cam و guided backpropagation است. در واقع grad-cam تنها می تواند نواحی مهم تصویر را به شکل تقریبی مشخص کند، اما جزئیات پیکسل به پیکسل تصویر را نشان نمی دهد و رزولوشن پایینی دارد. از سوی دیگر، guided backpropagation خروجی ای با جزئیات بالا می دهد اما فقط بر اهمیت کلی پیکسل ها تمرکز دارد و نمی تواند مشخص کند که کدام ناحیه تصویر برای پیش بینی یک کلاس خاص مهم است. با ترکیب این دو روش، guided grad-cam هم دقت فضایی بالای guided backpropagation را دارد و هم توانایی grad-cam در تمرکز روی کلاس هدف را حفظ می کند. به این ترتیب، این روش ضعف هر دو را برطرف کرده و تصویر نهایی هم دقیقا مناطقی از تصویر را که بیشترین تاثیر را در تصمیم کلاس دارند، مشخص می کند و هم جزئیات بصری خوبی می دهد. مثلا برای گربه، دقیقا نشان می دهد که خط های روی بدن آن در تصمیم گیر تاثیر داشته اند در حالی که با grad-cam به تنهایی فقط می توانیم بفهمیم که ناحیه بدن گربه در تصمیم گیری مهم بوده اما نمی توانم بفهمیم کدام ناحیه از آن بیشترین اهمیت را داشته است.

زیربخش دوم: پیاده سازی

قسمت الف و ب و ج: ابتدا مدل resnet18 و دیتاست را لود کردم. سپس برای پیاده سازی کلاس grad-cam، نیاز بود که ابتدا لایه مورد نظر که می خواهیم گرادیان ها را نسبت به feature map های خروجی آن بگیریم، به کلاس به عنوان ورودی بدهیم. این لایه در resnet18، feature map های خروجی از لایه 4 مدل هستند زیرا این لایه آخرین لایه قبل از لایه fully connected است و feature map های آن بهترین اطلاعات برای تصویر مورد نظر را دارند. سپس برای اجرای الگوریتم، نیاز داریم که خروجی آن لایه را نگه داریم (که همان feature map ها هستند) زیرا باید از logit کلاس هدف خروجی نسبت به همه آنها گرادیان بگیریم.

پس از گرادیان گرفتن هم نیاز داریم تا این گرادیان ها (که خودشان ماتریس با همان ابعاد feature map ها هستند) را نگه داریم تا با استفاده از آنها آلفا ها را حساب کنیم. پس یک بار از کلاس هدف backpropagation زدم و خروجی آن را در لایه هدف (که لایه 4 است) ذخیره کردم و سپس طبق مقاله، از هر ماتریس گرادیان خروجی میانگین گرفتم تا آلفا (وزن) متناظر با هر feature map بدست بیاید. در نهایت برای اینکه فقط تاثیرات مثبت را نگه داریم، یک relu روی آن اعمال می کنیم و آن را به آرایه 224*224 تغییر می دهیم که هر درایه آن بین 0 و 1 است (با نرمالایز کردن) که بتوانیم heatmap آن را روی تصویر اصلی قرار دهیم. برای آزمایش کردن هم 5 نمونه عکس که خود مدل آن را به درستی پیش بینی می کرد، انتخاب کردم و siliancy map را برای آنها نشان دادم که به صورت زیر شد:

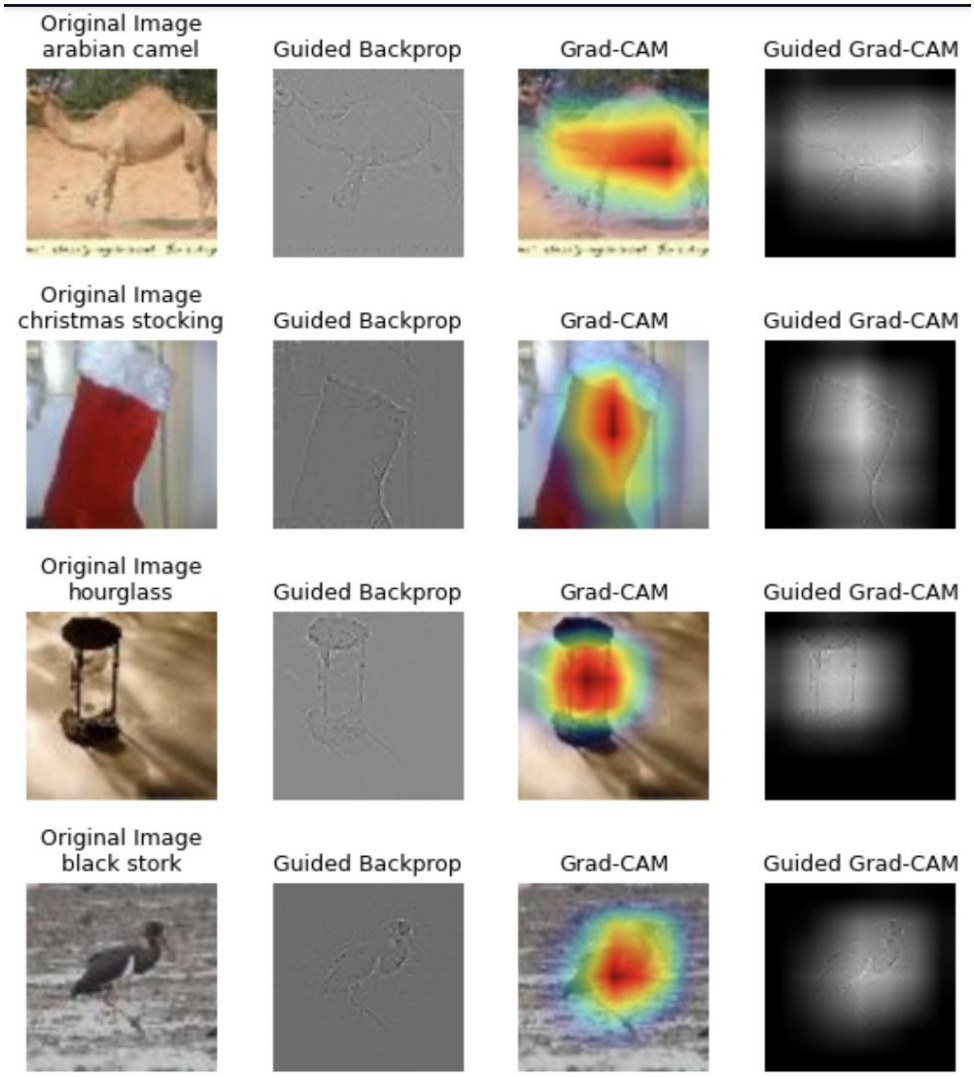


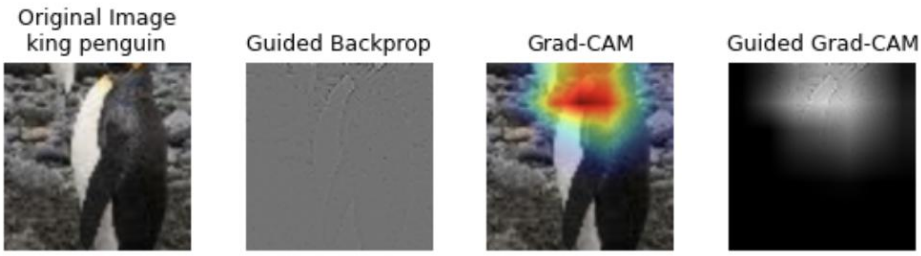
شکل 5. siliancy map برای 5 تصویر درست طبقه بندی شده

همانطور که دیده می شود، نقاط مهم تصاویر به درستی توسط grad-cam شناسایی شده اند.

قسمت د: سپس به پیاده سازی guided grad-cam پرداختم. ابتدا توضیحی درباره guided backpropagation می دهم. روش guided backpropagation یک روش مبتنی بر گرادیان برای مشخص کردن پیکسل های موثر در تصمیم مدل است. در حالت عادی، وقتی گرادیان خروجی یک کلاس نسبت به تصویر ورودی محاسبه می شود، هم اثر های مثبت و هم اثر های منفی در مسیر backpropagation گرادیان عبور می کنند. اما در guided backpropagation، هنگام عبور گرادیان از لایه های relu فقط گرادیان های مثبت اجازه عبور دارند و گرادیان های منفی صفر می شوند. بنابراین این روش فقط بخش هایی از تصویر را برجسته می کند که به صورت مثبت در افزایش امتیاز کلاس موردنظر نقش داشته اند. خروجی آن یک نقشه با جزئیات بالا از لبه ها و پیکسل های مهم تصویر است. اما guided backpropagation به تنهایی معمولا ناحیه کلی شی را به خوبی مشخص نمی کند و ممکن است جزئیات زیادی را نشان دهد که الزاما فقط مربوط

به کلاس هدف نیستند. به همین دلیل در guided grad-cam، خروجی guided backpropagation با خروجی grad-cam ترکیب می شود تا هم جزئیات پیکسلی حفظ شوند و هم فقط نواحی مرتبط با کلاس مورد نظر نمایش داده شوند. برای این کار ابتدا کلاس GuidedBackprop را نوشتم که الگوریتم را روی تصویر و کلاس داده شده اجرا می کند. در پیاده سازی آن، ابتدا خروجی تمام لایه های relu را ذخیره کردم تا در زمان backpropagation بتوان مسیر عبور گرادیان ها را کنترل کرد. سپس برای یک تصویر ورودی و کلاس هدف، امتیاز همان کلاس از خروجی مدل انتخاب شد و گرادیان آن نسبت به تصویر ورودی محاسبه شد. همانطور که گفتم تفاوت اصلی این پیاده سازی باbackpropagation معمولی این است که در لایه های relu، گرادیان های منفی حذف می شوند و فقط گرادیان های مثبت عبور می کنند. در نتیجه خروجی نهایی یک نقشه تصویری با جزئیات بالا است که نشان می دهد کدام پیکسل ها و لبه های تصویر به صورت مثبت در افزایش امتیاز کلاس هدف نقش داشته اند. سپس خروجی grad-cam را در خروجی guided backprop ضرب کردم تا خروجی guided grad-cam تولید شود. 5 نمونه از اجرا این الگوریتم ها به صورت زیر است:





شکل 6. saliency map روش های مختلف برای 5 تصویر درست طبقه بندی شده

همانطور که دیده می شود، guided grad-cam بهتر و دقیق تر از guided backprop توانسته محدوده شی موردنظر را مشخص کند و از سایر محدوده های بدون تاثیر در هر تصویر صرف نظر کند.

بخش دوم: FINERGRAD-CAM

زیربخش اول: پرسش و پاسخ

پاسخ سوال 1: این تمایز یعنی مشکل اصلی grad-cam در طبقه بندی fine-grained (که یعنی تفاوت تصاویر کلاس های مشابه ممکن است خیلی زیاد نباشد) لزوما در روش محاسبه saliency map نیست بلکه در هدفی است که از مدل می خواهیم توضیح دهد. grad-cam معمولا می پرسد «کدام نواحی تصویر باعث افزایش امتیاز کلاس هدف می شوند؟» اما در مسائل fine-grained ، کلاس ها بسیار شبیه اند و بسیاری از نواحی تصویر برای چند کلاس مشابه به طور مشترک مفید هستند. پس مثلا در طبقه بندی عکس های مربوط به پرندگان مختلف، grad-cam ممکن است کل بدن پرنده را برجسته کند زیرا این نواحی برای کلاس هدف مهم اند اما همان نواحی برای کلاس های مشابه هم مهم هستند و تفاوت واقعی را نشان نمی دهند. پس درست این نیست که «چرا این تصویر از کلاس هدف است؟» بلکه باید پرسید «چرا این تصویر از کلاس هدف است و نه از یک کلاس مشابه؟». پس با این توضیح این راه حل را ارائه دادند که هدف توضیح را از logit کلاس هدف به اختلاف logit کلاس هدف و کلاس مشابه تغییر دادند. در نتیجه، ویژگی های مشترک بین دو کلاس کم رنگ تر می شوند و نواحی متمایز کننده و متفاوت بیشتر دیده می شوند.

پاسخ سوال 2: spot-the-difference می گوید که ما در تشخیص fine-grained معمولا فقط به این فکر نمی کنیم که «این شی چه ویژگی هایی دارد؟» بلکه آن را با نمونه های بسیار مشابه مقایسه می کنیم و می پرسیم «چه چیزی این نمونه را از نمونه مشابه متمایز می کند؟». مثلا برای تشخیص دو نوع شبیه پرنده، دیدن کل بدن پرنده کافی نیست چون هر دو ممکن است بدن، شکل و رنگ کلی مشابهی داشته باشند و باید دید که ویژگی منحصر به فرد آنها نسبت به دیگری چیست. finer-cam هم می گوید به جای اینکه فقط

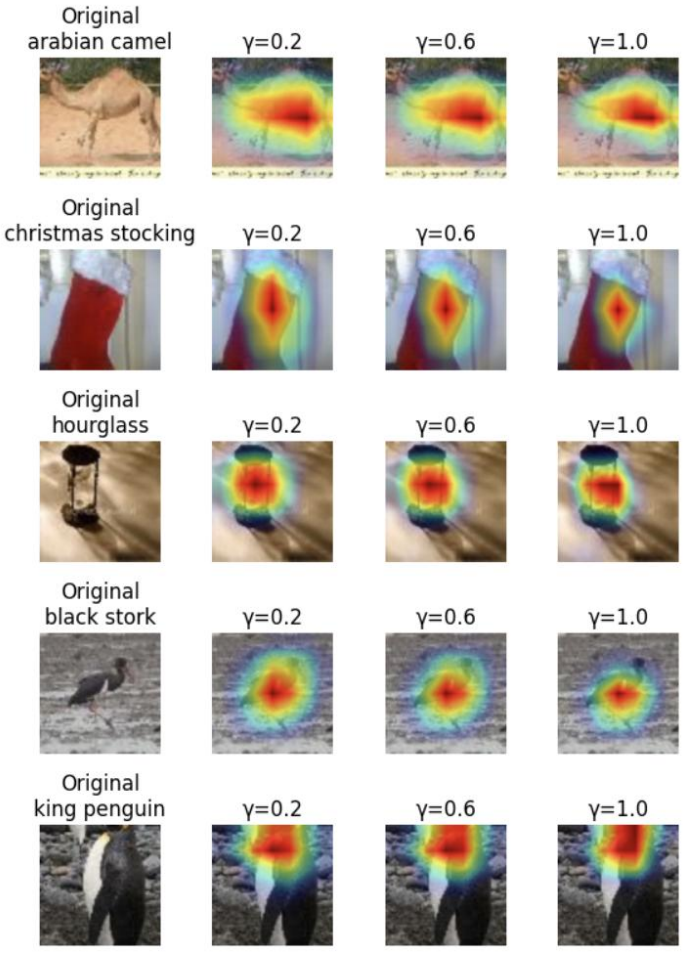
گرادیان مربوط به logit کلاس هدف y^c را محاسبه کند، هدف توضیح را به اختلاف بین کلاس هدف و یک (یا چند) کلاس مشابه تغییر می دهد که به صورت $y^c - \gamma y^d$ می شود. در نتیجه وزن اهمیت هر feature map به جای اینکه فقط α_k^c باشد، به صورت $\alpha_k^{c,d} = \alpha_k^c - \gamma \alpha_k^d$ محاسبه می شود. این یعنی ویژگی هایی که هم برای کلاس هدف و هم برای کلاس مشابه مفید اند کاهش می یابند اما ویژگی هایی که واقعا کلاس هدف را از کلاس مشابه جدا می کنند برجسته تر می شوند. بنابراین finer-cam از نظر مفهومی مانند مقایسه دو تصویر برای پیدا کردن تفاوت آنها است و از نظر ریاضی این مقایسه را با کم کردن سهم کلاس مشابه از سهم کلاس هدف در محاسبه saliency map پیاده سازی می کند.

پاسخ سوال 3: زیرا هدف توضیح همیشه یکسان نیست. گاهی کاربر می خواهد بداند مدل به طور کلی کدام شی را در تصویر برای تصمیم گیری دیده است. در این حالت مقدار کوچک تر γ مناسب تر است چون saliency map آن شبیه grad-cam می شود و بخش بزرگی از شی را نشان می دهد. اما در طبقه بندی fine-grained، هدف فقط دیدن کل شی نیست بلکه هدف پیدا کردن ویژگی های ظریفی است که یک کلاس را از کلاس های بسیار مشابه جدا می کند. در این حالت مقدار بزرگ تر γ ویژگی های مشترک با کلاس مشابه را بیشتر نادیده می گیرد و توجه را روی جزئیات متمایز کننده می گذارد. این قابلیت در حوزه هایی مانند زیست‌شناسی اهمیت زیادی دارد زیرا تشخیص گونه های نزدیک به هم معمولا بر اساس نشانه های بسیار کوچک و تخصصی انجام می شود و هر مقایسه ممکن است ویژگی متفاوتی را مهم کند. بنابراین finer-cam می تواند به متخصص کمک کند بفهمد مدل برای تمایز دو گونه دقیقا به چه صفاتی توجه کرده است.

پاسخ سوال 4: این قابلیت بررسی می کند که آیا ناحیه هایی که مدل برای تصمیم گیری استفاده کرده واقعا با ویژگی های درست مرتبط هستند یا نه. در حالت multi-modal مانند CLIP، به جای مقایسه فقط بین دو کلاس، می توان مفاهیم متنی یا صفات مشخص را هم مقایسه کرد. مثلا بررسی کرد آیا مدلی که یک پرنده را تشخیص داده، واقعا به صفات منحصر به فرد توجه کرده یا به نواحی نا مرتبط. در مقاله می گویند که finer-cam می تواند saliency map مربوط به تفاوت بین کلاس ها را با saliency map مربوط به صفات متنی مقایسه کند و سپس اگر این دو با هم هم راستا باشند، می توان گفت مدل احتمالا بر اساس ویژگی های درست و معنادار تصمیم گرفته است. اما اگر اختلاف زیادی وجود داشته باشد، ممکن است مدل به عوامل جانبی مانند پس زمینه یا ویژگی های اشتباه توجه کرده باشد. اهمیت این موضوع برای اعتماد به مدل های هوش مصنوعی این است که فقط درست بودن خروجی کافی نیست بلکه باید بدانیم مدل به دلیل درستی به جواب رسیده است مخصوصا در کاربردهای تخصصی و حساس که تصمیم مدل باید با دانش انسانی و ویژگی های واقعی مسئله قابل توجیه باشد.

زیربخش دوم: پیاده سازی

برای این قسمت از همان مدل و دیتاست و توابع نوشته شده در قسمت قبل استفاده کردم. در grad-cam، گرادیان score کلاس هدف نسبت به feature map های لایه آخر کانولوشن (که در اینجا لایه 4 resnet است) محاسبه می شود. اما در finerGrad-cam ، به جای score کلاس هدف، اختلاف بین score کلاس هدف و میانگین score سه کلاس مشابه در نظر گرفته شد (در اینجا d را مطابق صورت پروژه 3 گرفتم). یعنی برای هر تصویر، سه کلاس با بیشترین احتمال بعد از کلاس هدف به عنوان کلاس های مرجع استفاده شدند. همچنین پیاده سازی کلاس آن مشابه grad-cam بود با این تفاوت که برای score مقدار score تابع اصلی منهای میانگین score های 3 کلاس بعد از آن ضربدر γ را گذاشتم که به دلیل خطی بودن مشتق، پس از backpropagation نسبت به این مقدار، همان مقدار نهایی ای بدست می آید که در مقاله گفته شده. همچنین سه مقدار 0.2 و 0.6 و 1 را برای γ بررسی کردم. وقتی γ برابر صفر است، روش به grad-cam تبدیل می شود که نتایج آن در قسمت قبل برای این 5 تصویر موجود است. نتایج اجرای الگوریتم روی این تصاویر به صورت زیر است:



شکل 7. siliency map روش finergrad-cam برای 5 تصویر

در اینجا چون تصاویر به صورت ذاتی کوچک تر بودند و رزولوشن آنها کمتر از این مقادیری که به آنها resize کردم بوده، تفاوت بین map ها با γ متفاوت، خیلی زیاد نیست و همچنین احتمالا کلاس های نمونه در این 5 تصویر آنقدر متمایز از 3 کلاس دیگر بعد خود بوده اند که خیلی تفاوتی نکنند. اما مثلا برای تصویر پرنده می بینیم که تمرکز بر پایین تنه آن با افزایش γ بیشتر شده و برای عکس شتر هم همینطور. در کل با افزایش γ، اثر کلاس های مشابه بیشتر کم می شود و map نهایی بیشتر روی جزئیات متمایز کننده تمرکز می کند. 3 کلاس بعدی برای هر کدام از این کلاس ها به صورت زیر بودند:

Image 1
Target class: Arabian camel | prob:0.9955
Similar classes: ['sorrel', 'hartebeest', 'gazelle']

Image 2
Target class: Christmas stocking | prob:0.7810
Similar classes: ['overskirt', 'Windsor tie', 'velvet']

Image 3
Target class: hourglass | prob:0.5211
Similar classes: ['chime', 'gong', 'guillotine']

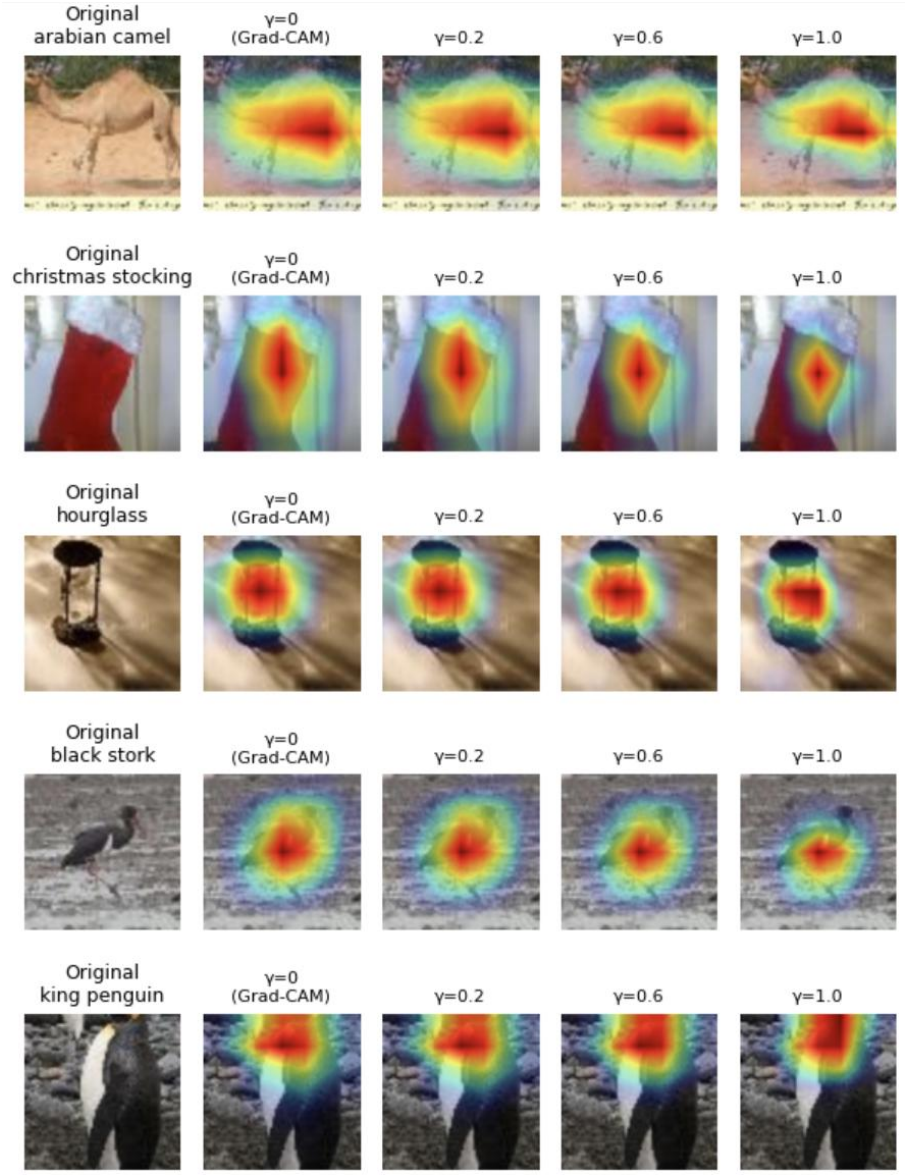
Image 4
Target class: black stork | prob:0.9202
Similar classes: ['black grouse', 'limpkin', 'red-breasted merganser']

Image 5
Target class: king penguin | prob:0.5923
Similar classes: ['tusker', 'Mexican hairless', 'curly-coated retriever']

بخش سوم: مقایسه

زیربخش اول: مقایسه دو روش

map های تولید شده برای 5 تصویر برای grad-cam و finergrad-cam (به ازای سه مقدار γ گفته شده) به صورت زیر هستند:



شکل 8. map های تولید شده به ازای هر دو روش

مشاهده می شود که grad-cam با مقدار $\gamma=0$ بیشتر ناحیه کلی شی را برجسته می کند زیرا فقط به دنبال بخش هایی است که امتیاز کلاس هدف را افزایش می دهند. اما در finergrad-cam با افزایشγ ، map ها کمی متمرکز تر می‌شوند و تلاش می کنند بخش هایی را نگه دارند که علاوه بر کمک به کلاس هدف، نسبت به کلاس های مشابه هم تمایزدهنده باشند. در تصویر شتر، چون مدل با احتمال بسیار بالا کلاس Arabian camel را تشخیص داده و کلاس‌های مرجع مانند sorrel ، hartebeestو gazelle (که در بخش قبل به آنها اشاره کردم) از نظر شکل کلی تا حدی به آن شباهت دارند، finergrad-cam همچنان روی بدن شتر متمرکز می‌ماند اما با افزایش γ ناحیه‌ی فعال کمی محدود تر می‌شود و بیشتر روی بخش مرکزی بدن قرار می‌گیرد. در تصویر Christmas stocking ، تفاوت واضح تری دیده می‌شود. grad-cam ناحیه‌ی نسبتا گسترده ای از جوراب و اطراف آن را فعال کرده اما با افزایش γ تمرکز map بیشتر روی بخش قرمز اصلی جوراب قرار می‌گیرد چون کلاس‌های دیگرoverskirt ، Windsor tieو velvet ممکن است از نظر بافت یا رنگ با تصویر شباهت داشته باشند و finergrad-cam سعی می‌کند ویژگی های مشترک را کمتر کند. در تصویرhourglass ، هر دو روش بیشتر روی بدنه ساعت شنی و قسمت مرکزی آن تمرکز دارند اما درγ های بزرگ تر تمرکز کمی منسجم تر و محدود تر می‌شود که نشان می دهد ویژگی متمایزکننده برای مدل بیشتر در ساختار مرکزی شی قرار دارد. در تصویر black stork هم grad-cam ناحیه ای نسبتا کلی از بدن پرنده و اطراف آن را نشان می دهد اما با افزایش γ تمرکز heatmap به بخش مشخص تری از بدن پرنده منتقل می‌شود که این منطقی است چون کلاس های مرجع همگی پرنده اند و finergrad-cam باید تفاوت های ظریف تری نسبت به پرندگان مشابه پیدا کند. در تصویر king penguin تغییرات چندان واضح نیست و ناحیه فعال همچنان روی بخش بالایی بدن و سر پنگوئن باقی می‌ماند. علت می‌تواند این باشد که کلاس های انتخاب‌شده مانند tusker ، Mexican hairless و curly-coated retriever شبیه پنگوئن نیستند بنابراین مقایسه finergrad-cam تفاوت ظریف زیادی ایجاد نمی کند. در مجموع، grad-cam برای نشان دادن ناحیه کلی مورد استفاده‌ی مدل مناسب تر است اما finergrad-cam زمانی مفید تر است که کلاس مشابه واقعا از نظر ظاهری نزدیک به کلاس هدف باشند. در این حالت با افزایشγ ، بخش های مشترک کاهش می یابند و map بیشتر به سمت جزئیات متمایزکننده حرکت می کند.

مراجع

[1] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

[2] Z. Zhang, J. Gu, A. Chowdhury, Z. Mai, D. Carlyn, T. Berger-Wolf, Y. Su, and W.-L. Chao, "Finer-CAM: Spotting the Difference Reveals Finer Details for Visual Explanation," *arXiv preprint arXiv:2501.11309*, 2025.

[3] D. W. Apley and J. Zhu, "Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 82, no. 4, pp. 1059–1086, 2020.

[4] R. Agarwal, N. Frosst, L. Melnick, X. Zhang, B. Lengerich, R. Caruana, and G. E. Hinton, "Neural Additive Models: Interpretable Machine Learning with Neural Nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 4699–4711.